FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 5 stretch: Where secure design and patterns come from

Software Developer Level 4



Patterns and pattern languages

IDEA 1

> **A pattern names a problem that keeps recurring and the core of a solution you can reuse your own way.**

Christopher Alexander (1977) wrote 253 patterns for towns and buildings; Gamma, Helm, Johnson and Vlissides (1994) brought the idea to object-oriented code with 23.

Why it matters: a shared name lets a team agree a design in minutes instead of drawing it every time.

Try this: name two patterns already at work in a framework you use, and the problem each one solves.

FIREBRAND LYCEUM EDUCATION GROUP

Saltzer and Schroeder's design principles

Written in 1975 for protecting information; many design flaws found today still break one of them.

1. **Economy of mechanism**
   Keep the design small enough to check.
2. **Fail-safe defaults**
   Deny unless access has been granted.
3. **Complete mediation**
   Check every access, every time.
4. **Open design**
   Security must not depend on a secret design.
5. **Separation of privilege**
   Need two conditions for the riskiest actions.
6. **Least privilege**
   Give each part only the rights its job needs.
7. **Least common mechanism**
   Share as little as possible between users.
8. **Psychological acceptability**
   Make the secure way the easy way.

Try this: which two principles does your revised design apply, and where?

FIREBRAND
LYCEUM EDUCATION GROUP

STRIDE: six ways a design can be attacked

Microsoft, 1999: six questions that turn 'is it secure?' into checks you can run.

Threat	Property it attacks	Ask of each component
Spoofing	Authentication	Can someone pretend to be another user or system?
Tampering	Integrity	Can data be changed in transit or at rest?
Repudiation	Non-repudiation	Could someone deny an action we cannot prove?
Information disclosure	Confidentiality	Can someone see data they should not?
Denial of service	Availability	Can someone stop it working for others?
Elevation of privilege	Authorisation	Can someone gain rights they were not given?

Try this: run STRIDE over your revised photo upload. Which threat is left?

Privacy by Design and UK GDPR Article 25

Privacy by Design (Cavoukian)

Proactive, not reactive

Privacy as the default setting

Privacy embedded into design

Full functionality: positive-sum, not zero-sum

End-to-end security, collection to deletion

Visibility and transparency

Respect for the user

UK GDPR Article 25

Data protection by design and by default is a legal duty in the UK.

Build measures in when you decide how data is processed, and by default process only what each purpose needs.

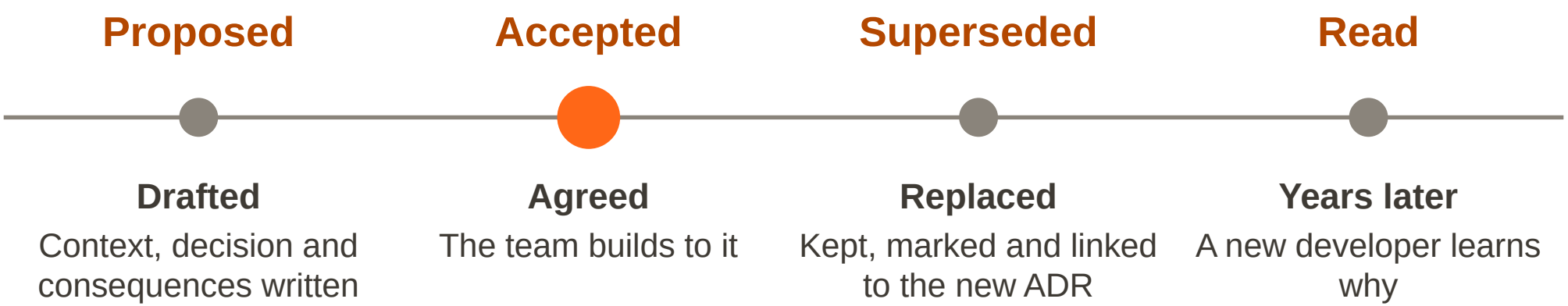
Why it matters: 'we will add privacy later' is not a lawful plan.

Try this: list three defaults that protect a tenant who never changes a setting.

FIREBRAND
LYCEUM EDUCATION GROUP

Architecture decision records as a log

Nygard (2011): short, numbered records that are never deleted.



Why it matters: a team can change course without losing the reason it started.

Try this: what later change would supersede your Activity 3 ADR?

FIREBRAND
LYCEUM EDUCATION GROUP

OWASP Top 10:2025 Insecure Design

A06

Insecure Design in the OWASP Top 10:2025

A flaw in the design itself: a missing or weak control. Perfect code cannot add a control nobody designed.

New in 2021 at fourth; sixth in 2025, as newer risks such as supply chain failures moved up.

OWASP's advice: threat modelling, secure design patterns and reference architectures, early.

Try this: pick a Kestrel Moor requirement that guards against insecure design. Which control does it demand?

FIREBRAND
LYCEUM EDUCATION GROUP

FIREBRAND
LYCEUM EDUCATION GROUP

Module 4: Software Design
Session 5 stretch: Where secure design and patterns come from



Software Developer Level 4

Optional self-study after Session 5, about 30 to 40 minutes. Six ideas sit behind today's session, one per slide: pattern languages, Saltzer and Schroeder's design principles, STRIDE threat modelling, Privacy by Design and UK GDPR Article 25, architecture decision records, and OWASP's Insecure Design category.

Each slide gives the idea in plain words, why it matters for a developer, and a 'Try this' question that applies it to Kestrel Moor Homes or to your own project. The notes under each slide hold the full reference, an answer guide and, where experts disagree, a note that the idea is contested.

How to use it: read a slide, answer its 'Try this' question in a few lines before reading the notes, then compare with the answer guide. Bring one answer to Session 6 if you want feedback.

Where a 'Try this' question asks about your own work: keep it general. No personal or customer data, credentials or confidential material, and no employer data in AI tools your employer has not approved.

Patterns and pattern languages

IDEA 1

> **A pattern names a problem that keeps recurring and the core of a solution you can reuse your own way.**

Christopher Alexander (1977) wrote 253 patterns for towns and buildings; Gamma, Helm, Johnson and Vlissides (1994) brought the idea to object-oriented code with 23.

Why it matters: a shared name lets a team agree a design in minutes instead of drawing it every time.

Try this: name two patterns already at work in a framework you use, and the problem each one solves.

FIREBRAND LYCEUM EDUCATION GROUP

2

References: Alexander, C., Ishikawa, S. and Silverstein, M. (1977) A Pattern Language: Towns, Buildings, Construction. Oxford University Press (253 patterns). Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley (23 patterns in creational, structural and behavioural groups).

The idea: Alexander's patterns each state a problem, discuss it and give the core of a solution 'in such a way that you can use this solution a million times over, without ever doing it the same way twice'. A pattern language links patterns so that larger ones make room for smaller ones.

Answer guide for 'Try this': good answers name a pattern and its problem, for example: event listeners in a web framework (Observer: many reactions to one event); middleware or filter chains (Chain of Responsibility: several handlers in turn); dependency injection containers (factories: controlled creation); a sort with a comparator (Strategy: interchangeable rules). Weak answers name a pattern with no problem.

Contested: some argue patterns mostly patch gaps in a language. Peter Norvig (1996) showed that 16 of the 23 patterns become invisible or simpler in dynamic languages such as Lisp or Dylan. The problems remain real; the code shape depends on the language.

Keep workplace examples general: no personal or customer data, credentials or confidential material, and no employer data in unapproved AI tools.

Saltzer and Schroeder's design principles

Written in 1975 for protecting information; many design flaws found today still break one of them.

1. **Economy of mechanism**
   Keep the design small enough to check.
2. **Fail-safe defaults**
   Deny unless access has been granted.
3. **Complete mediation**
   Check every access, every time.
4. **Open design**
   Security must not depend on a secret design.
5. **Separation of privilege**
   Need two conditions for the riskiest actions.
6. **Least privilege**
   Give each part only the rights its job needs.
7. **Least common mechanism**
   Share as little as possible between users.
8. **Psychological acceptability**
   Make the secure way the easy way.

Try this: which two principles does your revised design apply, and where?

FIREBRAND LYCEUM EDUCATION GROUP

3

Reference: Saltzer, J.H. and Schroeder, M.D. (1975) 'The Protection of Information in Computer Systems', Proceedings of the IEEE, 63(9), pp. 1278 to 1308. Section I sets out the eight design principles on this slide; it is readable without the hardware detail of Section II.

Why it matters: these principles explain most of the requirements in Case Brief Part 5. They are short enough to hold in your head during a design review, and they apply to any technology.

Answer guide for 'Try this': good answers name a principle and point to a specific class, link or operation in their own redesign, for example: least privilege (a method that returns only the fields one role needs); complete mediation (an access check on every request, not only at sign-in); separation of privilege (a password plus a second factor for the riskiest accounts); fail-safe defaults (access denied unless a role grants it); economy of mechanism (one small place where an access decision is made). Weak answers name a principle without pointing to anything in the design.

Not contested as principles, but note the tension the authors themselves raise: psychological acceptability can pull against the others, because a control people find too awkward gets bypassed.

STRIDE: six ways a design can be attacked

Microsoft, 1999: six questions that turn 'is it secure?' into checks you can run.

Threat	Property it attacks	Ask of each component
Spoofing	Authentication	Can someone pretend to be another user or system?
Tampering	Integrity	Can data be changed in transit or at rest?
Repudiation	Non-repudiation	Could someone deny an action we cannot prove?
Information disclosure	Confidentiality	Can someone see data they should not?
Denial of service	Availability	Can someone stop it working for others?
Elevation of privilege	Authorisation	Can someone gain rights they were not given?

Try this: run STRIDE over your revised photo upload. Which threat is left?

FIREBRAND LYCEUM EDUCATION GROUP

4

Reference: Kohnfelder, L. and Garg, P. (1999) 'The threats to our products', an internal Microsoft paper dated 1 April 1999; STRIDE later became part of Microsoft's Security Development Lifecycle and threat modelling tools. The original paper was internal for many years.

The idea: draw a data-flow diagram (people, processes, data stores and the flows between them, with trust boundaries marked) and ask the six STRIDE questions of each element. It is a prompt list, not a proof.

Why it matters: it is a structured way to do the 'trust boundaries' and 'failure paths' passes from Session 5, and it is widely used in industry and certification syllabuses.

Answer guide for 'Try this': good answers take each letter in turn and name the control in their own design that answers it, then say which letter is least covered. For any file upload, denial of service is often what remains (size limits, rate limits per user and a cap on files per request help), and repudiation is often forgotten (an audit log of who viewed or deleted each file). Credit any answer that names a real gap and a control that would close it.

Contested: STRIDE's completeness. Teams using it report few false positives but results vary with the team's experience, and joining components can create threats that no single element shows. Use it with, not instead of, thinking about the business.

Privacy by Design and UK GDPR Article 25

Privacy by Design (Cavoukian)

- Proactive, not reactive
- Privacy as the default setting
- Privacy embedded into design
- Full functionality: positive-sum, not zero-sum
- End-to-end security, collection to deletion
- Visibility and transparency
- Respect for the user

UK GDPR Article 25

Data protection by design and by default is a legal duty in the UK.

Build measures in when you decide how data is processed, and by default process only what each purpose needs.

Why it matters: 'we will add privacy later' is not a lawful plan.

Try this: list three defaults that protect a tenant who never changes a setting.

FIREBRAND LYCEUM EDUCATION GROUP

5

References: Cavoukian, A. (2009, revised 2011) Privacy by Design: The 7 Foundational Principles. Information and Privacy Commissioner of Ontario. UK GDPR Article 25, 'Data protection by design and by default', legislation.gov.uk. The Data (Use and Access) Act 2025 has amended parts of UK GDPR; always read the current text on legislation.gov.uk.

The idea: Cavoukian developed Privacy by Design in the 1990s. Article 25 turned part of it into law: measures such as pseudonymisation and data minimisation built in from the start, and by default personal data not made accessible to an indefinite number of people without the individual's intervention.

Answer guide for 'Try this' (any three, each a default the tenant does not have to switch on): update messages that say 'you have an update, sign in to view' rather than repair details; survey and marketing contact off unless the tenant opts in; location access off in a tenant app unless the tenant allows it; sessions that time out on a shared device; the lowest role for every new staff account. Credit any default that protects the tenant without action on their part.

Contested: Privacy by Design is criticised as vague and hard to turn into testable requirements, and 'positive-sum' can understate real trade-offs. Article 25 is law, but how far it requires a particular measure depends on cost, risk and context.

Architecture decision records as a log

Nygard (2011): short, numbered records that are never deleted.



Why it matters: a team can change course without losing the reason it started.

Try this: what later change would supersede your Activity 3 ADR?

FIREBRAND LYCEUM EDUCATION GROUP

6

Reference: Nygard, M. (2011) 'Documenting Architecture Decisions', Cognitect blog, 15 November 2011. Nygard's sections are title, context, decision, status and consequences; 'alternatives considered' was added by later templates such as MADR.

The idea: ADRs are numbered in order and kept with the code. A decision that is reversed is not deleted; it is marked Superseded and points to its replacement, because it still explains why the system looked the way it did. The consequences of one ADR often become the context of the next, which Nygard compares to Alexander's pattern language.

Why it matters: most of the cost of a design decision lands on people who were not there when it was made. A decision log is cheap to keep and answers 'why?' without a meeting.

Answer guide for 'Try this' on your own ADR: a move to a managed messaging platform that handles channels and retries; a requirement for guaranteed delivery order that simple events do not give; Kestrel Moor joining a shared notification service with other landlords; a change in the law or policy on how emergencies are communicated. Good answers name the trigger and what the new ADR would decide.

Mildly contested: how much to record. Some teams record every choice, which buries the important ones; Nygard's test is 'architecturally significant' decisions that affect structure, quality attributes, dependencies, interfaces or construction techniques.

OWASP Top 10:2025 Insecure Design

A06

Insecure Design in the OWASP Top 10:2025

A flaw in the design itself: a missing or weak control. Perfect code cannot add a control nobody designed.

New in 2021 at fourth; sixth in 2025, as newer risks such as supply chain failures moved up.

OWASP's advice: threat modelling, secure design patterns and reference architectures, early.

Try this: pick a Kestrel Moor requirement that guards against insecure design. Which control does it demand?

FIREBRAND LYCEUM EDUCATION GROUP

7

Reference: OWASP Top 10:2025, A06:2025 Insecure Design, top10.owasp.org. The category was introduced in the 2021 edition as A04:2021 and moved to sixth place in 2025, behind A02:2025 Security Misconfiguration and A03:2025 Software Supply Chain Failures among others.

The idea: OWASP separates insecure design from insecure implementation. A design can be implemented perfectly and still be unsafe if the control was never designed, for example no limit on what a user can upload, or no check on who may see a record. Notable weaknesses OWASP maps to this category include CWE-434 (unrestricted upload of a file with a dangerous type), CWE-269 (improper privilege management), CWE-501 (trust boundary violation) and CWE-522 (insufficiently protected credentials).

Why it matters: design review is the cheapest place to catch these, before any code exists. That is what Session 5 practised.

Answer guide for 'Try this': SEC-3 demands server-side validation of uploads (guards against CWE-434); SEC-2 demands role-scoped access (CWE-269); SEC-4 demands that photos are reachable only after sign-in (a trust boundary control, CWE-501); SEC-1 demands individual accounts with MFA (CWE-522); SEC-6 demands a designed response when something fails. Credit any requirement matched to the control it forces into the design.

Contested: the Top 10 is an awareness document, not a standard. Its ranking mixes data from contributed testing with a community survey, and the categories are broad. Use it to prompt thinking, and use a standard such as OWASP ASVS for testable requirements.